

Звіт

З дисципліни: “Обробка даних в інформаційних технологіях”

Лабораторна робота 3

Виконав: Климчук Богдан

Мета роботи: Удосконалення інформаційної системи шляхом реалізації повного набору CRUD-операцій та збереження інформації у файл.

Результати виконання завдань:

Нижче **Лістинги** та **Рисунки** лише частини Java

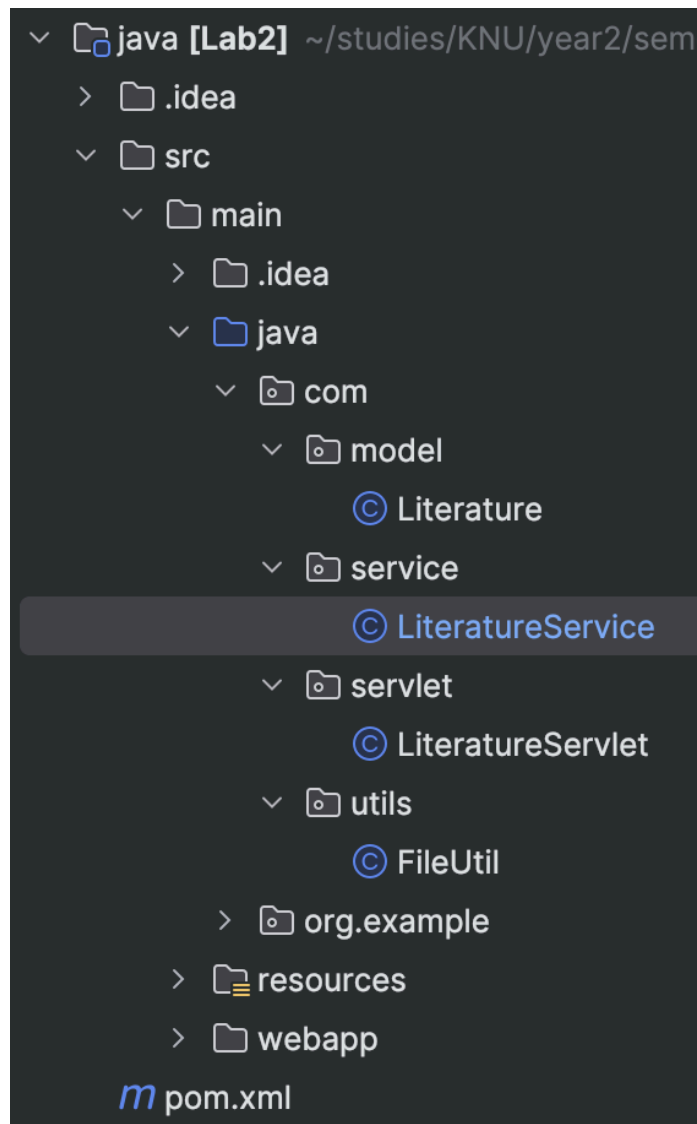


Рисунок 1 – структура проєкту Java

```

public LiteratureService() {
    literatureList = loadLiteratureList();
}

public List<Literature> getAllLiteratureList() {
    return literatureList;
}

public void addLiterature(Literature literature) {
    int newId = literatureList.isEmpty() ? 1 :
literatureList.get(literatureList.size() - 1).getId() + 1;
    literature.setId(newId);
    literatureList.add(literature);
    saveLiteratureList();
}

public boolean updatedLiterature(Literature updatedLiterature) {
    for (int i = 0; i < literatureList.size(); i++) {
        if (literatureList.get(i).getId() == updatedLiterature.getId()) {
            literatureList.set(i, updatedLiterature);
            saveLiteratureList();
            return true;
        }
    }
    return false;
}

public boolean deleteLiterature(int id) {
    boolean removed = literatureList.removeIf(l -> l.getId() == id);
    if (removed) saveLiteratureList();
    return removed;
}

private List<Literature> loadLiteratureList() {
    try {
        System.out.println("Читаю файл: " + FILE_PATH);
        String json = FileUtil.readFromFile(FILE_PATH);
        System.out.println("JSON з файлу: " + json);

        return new Gson().fromJson(json, new TypeToken<List<Literature>>()
{}.getType());
    } catch (IOException e) {
        return new ArrayList<>();
    }
}

private void saveLiteratureList() {
    try {
        String json = new Gson().toJson(literatureList);
        FileUtil.writeToFile(FILE_PATH, json);
    } catch (IOException e) {
        e.printStackTrace();
    }
}

```

Лістинг 1 – вміст файлу LiteratureService

```

        @WebServlet("/literature")
        public class LiteratureServlet extends HttpServlet {
            private LiteratureService literatureService = new LiteratureService();
            private Gson gson = new Gson();

            @Override
            protected void doGet(HttpServletRequest req, HttpServletResponse resp)
                throws IOException {
                List<Literature> literatureList =
                    literatureService.getAllLiteratureList();
                System.out.println("Книги у відповіді: " + literatureList.size()); //
                // Логування
                resp.setContentType("application/json");
                resp.getWriter().write(gson.toJson(literatureList));
            }

            @Override
            protected void doPost(HttpServletRequest req, HttpServletResponse resp)
                throws IOException {
                BufferedReader reader = req.getReader();
                Literature newLiterature = gson.fromJson(reader, Literature.class);
                literatureService.addLiterature(newLiterature);
                resp.setStatus(HttpServletResponse.SC_CREATED);
            }

            @Override
            protected void doPut(HttpServletRequest req, HttpServletResponse resp)
                throws IOException {
                BufferedReader reader = req.getReader();
                Literature updatedLiterature = gson.fromJson(reader,
                    Literature.class);
                if (literatureService.updateLiterature(updatedLiterature)) {
                    resp.setStatus(HttpServletResponse.SC_OK);
                } else {
                    resp.setStatus(HttpServletResponse.SC_NOT_FOUND);
                }
            }

            @Override
            protected void delete(HttpServletRequest req, HttpServletResponse resp)
                throws IOException {
                int id = Integer.parseInt(req.getParameter("id"));
                if (literatureService.deleteLiterature(id)) {
                    resp.setStatus(HttpServletResponse.SC_OK);
                } else {
                    resp.setStatus(HttpServletResponse.SC_NOT_FOUND);
                }
            }
        }

```

ЛІСТИНГ 2 – ВМІСТ ФАЙЛУ LiteratureServlet

```

public class Literature {
    private int id;
    private String title;
    private String author;
    private int pages;
    private String imageUrl;
    private boolean isHardcover;

    public Literature(int id, String title, String author, int pages, String
imageUrl, boolean isHardcover) {
        this.title = title;
        this.author = author;
        this.pages = pages;
        this.imageUrl = imageUrl;
        this.isHardcover = isHardcover;
        this.id = id;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }
}

```

Лістинг 3 – вміст файлу Literature

```

public class FileUtil {
    public static String readFromFile( String filePath ) throws IOException
{
        System.out.println("Читаю файл: " + filePath);
        return new String(Files.readAllBytes(Paths.get(filePath)));
    }

    public static void writeToFile( String filePath, String data ) throws
IOException {
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(filePath))) {
            writer.write(data);
        }
    }
}

```

Лістинг 4 – вміст файлу FileUtil

Тепер перейдемо до реалізації цієї роботи на Vue.js

```
<template>
  <div>
    <h1> Manage books system</h1>
    <h3>{{isEditing ? "Edit books list" : "Add new book"}}</h3>
    <form @submit.prevent="isEditing ? updateBook() : addBook()">
      <input v-model="currentBook.title" type="text"
placeholder="Title">
      <input v-model="currentBook.author" type="text"
placeholder="Author">
      <input v-model="currentBook.pages" type="number" placeholder="12">
      <input v-model="currentBook.imageUrl" type="text"
placeholder="Book image">
      <input v-model="currentBook.isHardcover" type="text"
placeholder="Book hardcover">
      <button type="submit">{{isEditing ? "Edit" : "Add"}}</button>
      <button type="button" v-if="isEditing"
@click="cancelEdit()">Cancel</button>
    </form>

    <h1>List of the books</h1>
    <ul>
      <li v-for="book in books" :key="book.id">
        <h2>{{ book.title }}</h2>
        <h3>{{ book.author }}</h3>
        <p>{{ book.pages }}</p>
        <img :src='book.imageUrl' :style="{width: '40px', height:
'40px'}"/>
        <p>{{ book.isHardcover ? "hardcover" : "paperback" }}</p>
        <button @click="editBook(book)">Edit</button>
        <button @click="deleteBook(book.id)">Remove</button>
      </li>
    </ul>
  </div>
</template>

<script setup>
```

```
import { ref, onMounted } from 'vue';
const books = ref(null);
const currentBook = ref({id: null, title: "", author: "", pages: null,
imageUrl: "", isHardcover: null})
const isEditing = ref(false);

const fetchBooks = async () => {
  try {
    const response = await fetch("/literature");
    books.value = await response.json();
    console.log("Отримані книги:", books.value);
  } catch (e) {
    console.error("Помилка при отриманні даних:", e);
  }
};

const addBook = async () => {
  try {
    const response = await fetch('./literature', {
      method: "POST",
      headers: {'Content-Type': "application/json"},
      body: JSON.stringify(currentBook.value),
    })

    if ( response.ok ) {
      fetchBooks();
      resetForm();
    }
  } catch (error) {
    console.error("Error during adding the book", error)
  }
}

const editBook = (book) => {
  currentBook.value = { ...book }
  isEditing.value = true;
}

const updateBook = async () => {
```

```
try {
  const response = await fetch('./literature', {
    method: "PUT",
    headers: {'Content-Type': "application/json"},
    body: JSON.stringify(currentBook.value),
  })

  if ( response.ok ) {
    fetchBooks();
    resetForm();
  }
} catch (error) {
  console.error("Error during updating the book", error)
}

const deleteBook = async (id) => {
  try {
    const response = await fetch(`./literature?id=${id}`, { method:
"DELETE" })

    if ( response.ok ) {
      fetchBooks();
    }
  } catch (e) {
    console.error("Error during deleting the book", e)
  }
}

const resetForm = () => {
  currentBook.value = { id: null, title: "", author: "", pages: null,
imageUrl: "", isHardcover: null }
  isEditing.value = false;
}

const cancelEdit = () => {
  resetForm();
}
```

```
onMounted(fetchBooks)
</script>
```

Лістинг 5 — Вміст файлу App.vue

У файлі *vite.config.js* було змінено поле “target”, оскільки у файлу pom.xml було змінено назву збірки

```
export default defineConfig({
  plugins: [vue()],
  build: {
    outDir: 'dist',
    emptyOutDir: true
  },
  server: {
    proxy: {
      '/literature': {
        target: 'http://localhost:8080/Lab3-3.0-SNAPSHOT',
        changeOrigin: true,
        secure: false
      }
    }
  }
});
```

Лістинг 6 – Вміст файлу vite.config.js

Висновок: в ході лабораторної роботи було удосконалено інформаційну систему шляхом реалізації повного набору CRUD-операцій та збереження інформації у файл.

Відповіді на запитання для самоперевірки:

1. Як реалізовані CRUD-операції у бекенд-частині проєкту?
 2. Як відбувається збереження даних у файл?
 3. Як працює оновлення даних у фронтенді без перезавантаження сторінки?
 4. Як обробляються помилки при взаємодії між фронтендом і бекендом?
 5. Як забезпечити коректну роботу DELETE-запиту у сервлетах?
 6. Як правильно працювати з Git-гілками під час розробки?
- 1) CRUD (Create, Read, Update, Delete) реалізовані через *LiteratureServlet*, який працює із сервісним класом *LiteratureService*.

Create	POST /literature	doPost()	addLiterature(Literature literature)
Read	GET /literature	doGet()	getAllLiteratureList()
Update	PUT /literature	doPut()	updateLiterature(Literature literature)
Delete	DELETE /literature?id={id}	doDelete()	deleteLiterature(int id)

2) Дані книг зберігаються у JSON-файл за допомогою **FileUtil**.

```
private void saveLiteratureList() {
    try {
        String json = new Gson().toJson(literatureList);
        FileUtil.writeToFile(FILE_PATH, json);
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

3) Фронтенд використовує **Vue.js** та метод **fetchBooks()**, який оновлює список книг після кожної CRUD-операції.

Механізм оновлення

1. Користувач додає/редагує/видаляє книгу.
 2. Відправляється відповідний HTTP-запит (POST, PUT, DELETE).
 3. Після успішного запиту викликається **fetchBooks()**, який оновлює **books.value**.
 4. **Vue** автоматично оновлює **DOM**, оскільки **books.value** – це реактивна змінна.
- 4)

Обробка помилок на фронтенді При помилці в **fetch** використовується **try...catch**, а **console.error()** виводить повідомлення у консоль браузера.

Обробка помилок на бекенді

Сервлет обробляє винятки та повертає коректний HTTP-статус.

5)

Проблеми, які можуть виникнути

1. Немає параметра **id** у запиті → викликає **NumberFormatException**.
2. Книги з таким **id** не існує → повертає **404 Not Found**.
3. Сервер не обробляє **DELETE**-запити (якщо неправильно налаштований метод **doDelete**).

Нам потрібно:

Перевіряти, чи передано **id**.

Обробляти некоректний **id** (наприклад, якщо замість числа передано **abc**).

Якщо книга знайдена і видалена → **200 OK**, якщо ні → **404 Not Found**.

6)

1. Створювати нову гілку для виконання нового завдання
2. Видаляти застарілі гілки для того, щоб не “засмічувати” репозиторій і локальне середовище